

Liquidación semanal de una empresa de mensajería

Parte A. Diseño

1. Separación modular

El programa se divide en módulos con una única responsabilidad. Cada uno vive en su propio archivo

`.exs` y `main.exs` los carga en orden de dependencia con `Code.require_file`.

Módulo	Responsabilidad	Tipo
Datos	Contiene únicamente los datos del grupo (repartidores, zonas, servicios).	Puro (solo devuelve constantes)
Formato	Da formato a pesos, decimales y alineación de texto.	Puro
Validacion	Aplica las 5 reglas con <code>with</code> y clasifica en válidos / rechazados.	Puro
Liquidacion	Valor del servicio, bonificación, alquiler, detalle por día y neto.	Puro
Reportes	R1–R8. Cada reporte tiene una parte pura (calcula) y una función <code>rN</code> que imprime.	Mixto (cálculo puro, impresión impura)
Entrada	Servicio adicional y comprobante. Análisis del texto puro; lectura e impresión impuras.	Mixto
Investigacion	<code>ranking/2</code> , <code>Map.merge/3</code> y mediciones con <code>:timer.tc/1</code> .	Mixto
Main	Orquesta: carga datos, valida, pide el servicio, imprime reportes y comprobante.	Impuro

2. Colecciones elegidas y justificación

Colección	Dónde se usa	Por qué se eligió
Lista de mapas	Repartidores, zonas y servicios (<code>Datos</code>); liquidaciones; detalle por día.	Los datos llegan como listas y mantienen un orden. Cada registro tiene campos con nombre y distinto tipo, así que un mapa los representa bien sin necesidad de <code>deconstruct</code>

		(prohibido).
Mapa indexado por clave	<code>codigo => repartidor</code> <code>r y id => zona.</code>	Validar un servicio exige saber si el repartidor y la zona existen. Con un mapa la búsqueda no recorre toda la colección (medido en C3: unas 2.9 veces más rápido que buscar en lista).
Mapa de acumulación	Km por día (<code>dia => km</code>), conteo de rechazos por motivo, resultados de <code>group_by</code> .	Agrupar por una clave y sumar es el uso natural de un mapa; además <code>Map.merge / 3</code> trabaja sobre ellos.
Tupla	<code>{:ok, servicio}</code> , <code>{:error, motivo}</code> , <code>{servicio, motivo}</code> , <code>{validos, rechazados}</code> .	Tamaño fijo y significado por posición. El resultado etiquetado reemplaza a <code>try / rescue</code> .
Átomo	Motivos de rechazo (<code>:dia_invalido</code> , etc.).	Valores constantes, comparables en tiempo constante y legibles.
MapSet	R8 (zonas cubiertas por un repartidor).	La pregunta es de conjuntos: ¿el conjunto de zonas de la empresa es subconjunto del que cubrió el repartidor? <code>MapSet.subset? / 2</code> lo responde sin duplicados.
Keyword list	Opciones de <code>ranking / 2</code> (<code>campo:</code> , <code>orden:</code> , <code>limite:</code>).	Es la convención de Elixir para opciones: lista ordenada de pares con átomos como clave y valores por defecto.
Atributos de módulo	Tarifa, meta, días, km máximo, bonificación, alquiler.	El enunciado exige definir los parámetros como atributos del módulo correspondiente.

3. Estructuras de control

Estructura	Dónde	Justificación
<code>with</code>	<code>Validacion.validar/3</code> y análisis del texto del servicio adicional.	Encadena verificaciones que devuelven <code>{:ok, _}</code> y se detiene en el primer <code>{:error, _}</code> , que es exactamente la regla de “solo se registra el primer error en el orden dado”.
<code>cond</code>	<code>Liquidacion.porcentaje_ajuste/1</code> (retraso).	Son rangos ordenados (≤ 0 , ≤ 10 , ≤ 30 , resto); con <code>cond</code> cada condición descarta a la anterior.
<code>case</code>	Validación de día, km y retraso; resultado de <code>ranking/2</code> , R7 y comprobante.	Se decide según la forma del valor (<code>{:ok, _}</code> , <code>:error</code> , lista vacía).
Cláusulas múltiples con guardas	<code>bonificacion/1</code> , <code>alquiler/2</code> , <code>procesar_entrada/3</code> , <code>costo_por_km/2</code> .	Cada caso queda como una función corta con un nombre claro; la guarda expresa la condición (<code>km >= 80</code>).
<code>if</code>	Existencia de repartidor/zona; marcar meta alcanzada.	Solo hay dos ramas, así que no hace falta nada más elaborado.
<code>Enum</code> y <code>for</code>	Todos los recorridos.	El enunciado prohíbe la recursividad. <code>for</code> sirve para filtrar y transformar a la vez (por ejemplo, separar válidos de rechazados).

4. Funciones puras e impuras

Una función es **pura** si con los mismos argumentos devuelve siempre el mismo resultado y no hace entrada/salida ni modifica nada externo. Es **impura** si lee del teclado, imprime o depende del tiempo.

Pura	Impura
<code>Validacion.*</code> (validar, clasificar, cada regla)	<code>Main.main/0</code>
<code>Liquidacion.*</code> (valor, bonificación, alquiler, liquidar)	<code>Reportes.r1 ... r8</code> (imprimen con <code>IO.puts</code>)

Formato.*	E n t r a d a . l e e r _ l i n e a , s o l i c i t a r _ s e r v i c i o _ a d i c i o n a l , s o l i c i t a r _ c o m p r o b a n t e , i m p r i m i r _ c o m p r o b a n t e
Cálculos de Reportes: contar_ rechazos, kilometros_por_zo na, kilometros_por_dia, lidere s_por_dia, puntualidad, total_p agado, en_todas_las_zonas	Investigacion.mostrar_ranking, mostrar_combinacion
Entrada.parsear_servicio, pars ear_numero, procesar_entrada, a rmar_comprobante	Investigacion.mediciones (usa:t imer.tc, depende del reloj)
Investigacion.ranking/2, combina r_km_diarios, Datos.* (solo devuelve constantes)	

Criterio de diseño: la lógica de negocio está en funciones puras y la entrada/salida queda en una capa delgada. Así los cálculos se pueden probar sin teclado ni pantalla, y cada reporte puede probarse por separado con datos pequeños.

5. Supuestos de negocio

- El valor de cada servicio se redondea al peso entero más cercano (`round / 1`), para que todo el dinero del programa sea entero y no se acumulen errores de punto flotante. Los porcentajes se calculan con enteros (108, 100, 90, 75).
- Los servicios del archivo de datos y el servicio adicional pasan por la misma validación. Un servicio adicional rechazado por una regla aparece en R1; uno rechazado por formato no, porque nunca llegó a ser un servicio.
- Los códigos se comparan exactamente como se escriben: `m 0 1` no es `M 0 1` (el conjunto de datos incluye un caso así a propósito).
- En R5, un día empatado cuenta como primer lugar para todos los empatados.

Explicación de R6: puntualidad ponderada

El retraso ponderado es $\text{suma}(\text{retraso} \times \text{km}) / \text{suma}(\text{km})$. El promedio simple es $\text{suma}(\text{retraso}) / \text{cantidad de servicios}$.

1. ¿Por qué puede ser distinto del promedio simple?

El promedio simple trata todos los servicios como si valieran lo mismo. El ponderado multiplica cada retraso por los kilómetros del servicio, es decir, es un promedio donde cada servicio pesa según su distancia. Los dos coinciden cuando todos los servicios tienen los mismos kilómetros; en cualquier otro caso suelen diferir. Si los retrasos grandes ocurren en servicios cortos, el ponderado queda **por debajo** del simple; si ocurren en servicios largos, queda **por encima**.

2. Ejemplo con nuestros datos: Valeria Duque (M09)

M09 tiene 12 servicios válidos. Su servicio del día 6 en Z4 recorrió solo 17 km pero tuvo 180 minutos de retraso:

Día	Zona	Km	Retraso (min)	Retraso × km
1	Z2	15	28	420
1	Z3	27	10	270
2	Z2	32.5	22	715
2	Z3	43	-5	-215
3	Z3	37	15	555
3	Z4	25.5	45	1147.5
4	Z3	28	12	336
4	Z4	43	-12	-516
5	Z4	29.5	2	59
5	Z1	21.5	8	172
6	Z4	17	180	3060
6	Z1	33	8	264
Total		352	313	6267,5

• **Promedio simple** = $313 / 12 = 26,08$ min.

• **Promedio ponderado** = $6267,5 / 352 = 17,81$ min.

La diferencia es de más de 8 minutos. El servicio de 180 minutos pesa $1/12$ (8,3 %) en el promedio simple, pero solo $17/352 = 4,8$ % en el ponderado, porque fue un servicio corto. Por eso el simple castiga más a M09 de lo que merece.

Otro caso: David López (M02), el ganador de R6, tiene simple $-3,73$ y ponderado $-2,16$. Sus adelantos más grandes (-25 y -18 min) ocurrieron en servicios de 15 a 34 km; el ponderado los atenúa un poco.

3. ¿Por qué tiene sentido darle más peso a un servicio largo?

Llegar a tiempo en un trayecto de 40 km es más difícil y más valioso que en uno de 5 km: hay más tráfico, más posibilidades de imprevistos y más tiempo comprometido del repartidor. Además, el cliente y la empresa pagan más por ese servicio (el valor es proporcional a los km), así que su puntualidad afecta una mayor parte del negocio. Un retraso de 30 minutos en un servicio corto no debería pesar igual que uno en un servicio largo; la medida ponderada refleja cuánto del trabajo real del repartidor salió puntual. También evita que alguien mejore su promedio simple intercalando muchos servicios cortos y fáciles.

Parte C. Investigación

C1. Keyword lists y la función `ranking / 2`

Una **keyword list** es una lista de tuplas `{ átomo, valor }`, por ejemplo `[ campo : : neto, limite : 3 ]`. Se usa para pasar opciones porque: (a) las opciones son opcionales y se pueden omitir; (b) se escriben sin llaves cuando son el último argumento; (c) conservan el orden y permiten claves repetidas; y (d) es la convención del lenguaje, de modo que cualquier programador de Elixir la reconoce.

Esta es la definición:

```
@opciones_por_defecto [campo: :kilometros, orden: :desc, limite: 5]

def ranking(coleccion, opciones \\ []) do
  with {:ok, config} <- Keyword.validate(opciones, @opciones_por_defecto),
       campo = Keyword.get(config, :campo),
       orden = Keyword.get(config, :orden),
       limite = Keyword.get(config, :limite),
       :ok <- validar_campo(coleccion, campo),
       :ok <- validar_orden(orden),
       :ok <- validar_limite(limite) do
    ordenada = Enum.sort_by(coleccion, fn e -> Map.get(e, campo) end, orden)
    {:ok, Enum.take(ordenada, limite)}
  else
    {:error, claves} when is_list(claves) -> {:error, {:opciones_desconocidas, claves}}
    {:error, motivo} -> {:error, motivo}
  end
end
```

```
end
```

`Keyword.validate/2` hace dos cosas a la vez: rechaza opciones que no existen y completa los valores por defecto. El resultado es siempre `{:ok, lista}` o `{:error, motivo}`, sin `try/rescue`. Los resultados reales con las liquidaciones del grupo aparecen en la sección de salida: `campo: :neto, limite:3` devuelve Ana Torres, Camila Restrepo y Valeria Duque; `orden: :arriba` devuelve `orden_invalido`; y `color: :rojo` devuelve `{:opciones_desconocidas, [:color]}`.

C2. Combinación de mapas con `Map.merge/3`

```
def combinar_km_diarios(km_propios, km_aliada) do
  Map.merge(km_propios, km_aliada, fn _dia, propio, alia
do -> propio + aliado end)end
```

Con el mapa propio de R3 y el de la empresa aliada el resultado real es:

```
Propios (R3):  %{1 => 540.0, 2 => 430.5, 3 => 515.5, 4 => 395.0, 5 => 560.0, 6 => 500.0}
Empresa aliada: %{1 => 580.5, 2 => 430, 3 => 510, 5 => 625, 7 => 180}
Map.merge/2:   %{1 => 580.5, 2 => 430, 3 => 510, 4 => 395.0, 5 => 625, 6 => 500.0, 7 => 180}
Map.merge/3 (suma): %{1 => 1120.5, 2 => 860.5, 3 => 1025.5, 4 => 395.0, 5 => 1185.0, 6 => 500.0, 7 => 180}
```

1. ¿Qué produce `Map.merge/2`? Cuando una clave está en los dos mapas, se queda con el valor del *segundo* mapa y descarta el del primero. En los días 1, 2, 3 y 5 el resultado serían los km de la empresa aliada (580,5; 430; 510; 625) y se perderían los nuestros (540; 430,5; 515,5; 560).

2. ¿Por qué no resuelve el problema? El problema pide sumar los kilómetros de ambas empresas en los días repetidos. `merge/2` los sobrescribe, así que el total diario quedaría subestimado y se perdería información de la empresa propia sin ningún aviso. `Map.merge/3` recibe una función de resolución de conflictos `fn clave, valor1, valor2 -> ... end` que se ejecuta solo para las claves repetidas; ahí se hace `propio + aliado`.

3. ¿Qué pasa con el día 7? Solo existe en el mapa de la empresa aliada (nuestro programa solo opera días 1 a 6). Como no está en conflicto, la función de suma *no se llama* y el par `7 => 180` se copia tal cual al resultado. Lo mismo ocurre con los días 4 y 6, que solo están en nuestro mapa. Hay que tener presente que el día 7 no es un día de operación válido para nosotros, así que el resultado mezcla un día que nuestras reglas no reconocen; la función no lo filtra porque el enunciado no lo pide.

C3. Mediciones con `:timer.tc/1`

`:timer.tc(fn -> ... end)` devuelve `{microsegundos, resultado}`. Se midieron cinco cosas con los datos del grupo (116 servicios, 12 repartidores). Los valores cambian un poco entre ejecuciones; estos son los de la salida que se anexa.

#	Medición	Resultado
1	Validar los 116 servicios, una sola vez	19 μ s
2	Validar 1000 veces (promedio por vez)	20.24 μ s
3	Liquidar a los 12 repartidores, 1000 veces (promedio por vez)	34.85 μ s
4	100 000 búsquedas del último repartidor en lista (<code>Enum.a_ny?</code>)	23987 μ s
5	100 000 búsquedas del último repartidor en mapa (<code>Map.has_key?</code>)	8181 μ s

Explicación. (1 y 2) Validar todo el conjunto toma unas decenas de microsegundos: es un cálculo muy liviano. Una sola medición (19 μ s) es poco confiable porque el reloj, el planificador de la máquina virtual y la caché la afectan; por eso se repite 1000 veces y se promedia (20.24 μ s). (3) Liquidar cuesta algo más que validar porque agrupa por repartidor y por día y calcula el valor de cada servicio. (4 y 5) Buscar en la lista recorre elementos hasta encontrar el código (se eligió el *último* repartidor, el peor caso), mientras que el mapa accede por clave. En esta prueba el mapa fue unas **2.9 veces** más rápido. Con solo 12 repartidores la diferencia absoluta es mínima (menos de un microsegundo por búsqueda), pero la lista crece de forma lineal con el número de elementos y el mapa casi no. Por eso la validación recibe mapas indexados y no listas.

Salida completa del programa

Ejecución con los datos del grupo: se presionó Enter para omitir el servicio adicional y se pidió el comprobante de M 0 3 . Datos: 12 repartidores (6 con bicicleta), 4 zonas, 99 servicios válidos y 17 inválidos (3 por :repartidor_desconocido, 2 por :zona_desconocida y 4 por cada uno de los otros tres motivos).

```
Ingrese un servicio
adicional(repartidor;
zona;dia;kilometros;re
traso) o Enter para omi
tir:
```

> No se ingresó ningún servicio adicional.

=== R1. Servicios rechazados ===

```
:repartidor_desconocido <- %{dia: 2, kilometros: 20, repartidor: "M99", retr
aso: 5, zona: "Z1"}
:repartidor_desconocido <- %{dia: 3, kilometros: 18, repartidor: "m01", retr
aso: 0, zona: "Z2"}
:repartidor_desconocido <- %{dia: 9, kilometros: 99, repartidor: "M98", retr
aso: 999, zona: "Z8"}
:zona_desconocida <- %{dia: 1, kilometros: 15, repartidor: "M02", retraso:
3, zona: "Z9"}
:zona_desconocida <- %{dia: 2, kilometros: 22, repartidor: "M04", retraso:
5, zona: nil}
:dia_invalido <- %{dia: 7, kilometros: 20, repartidor: "M12", retraso: 5, z
ona: "Z1"}
:dia_invalido <- %{dia: "3", kilometros: 25, repartidor: "M06", retraso: 5,
zona: "Z2"}
:dia_invalido <- %{dia: 2.5, kilometros: 30, repartidor: "M07", retraso: 0,
zona: "Z3"}
:dia_invalido <- %{dia: 0, kilometros: -1, repartidor: "M02", retraso: 500,
zona: "Z1"}
:kilometros_fuera_de_rango <- %{dia: 4, kilometros: 46, repartidor: "M08",
retraso: 5, zona: "Z4"}
:kilometros_fuera_de_rango <- %{dia: 5, kilometros: 0, repartidor: "M09", r
etraso: 0, zona: "Z1"}
:kilometros_fuera_de_rango <- %{dia: 2, kilometros: "abc", repartidor: "M0
1", retraso: 3, zona: "Z2"}
:kilometros_fuera_de_rango <- %{dia: 6, kilometros: -5, repartidor: "M03",
retraso: 10, zona: "Z3"}
:retraso_invalido <- %{dia: 1, kilometros: 20, repartidor: "M05", retraso:
181, zona: "Z4"}
:retraso_invalido <- %{dia: 3, kilometros: 12, repartidor: "M06", retraso:
-31, zona: "Z1"}
:retraso_invalido <- %{dia: 4, kilometros: 18, repartidor: "M07", retraso:
```

```

nil, zona: "Z2"}
:retraso_invalido <- %{dia: 5, kilometros: 22, reparti
dor: "M08", retraso: "x", zona: "Z3"} Cantidad de rech
azos por motivo:

```

```

:repartidor_desconocido      3
:zona_desconocida            2
:dia_invalido                 4
:kilometros_fuera_de_rango    4
:retraso_invalido            4
Total                         17

```

```

=== R2. Kilómetros por zona y densidad de recorrido ===
Z1 Centro 679.0 km  área  6.50 km²  densi
dad 104.46 km/km² Z3 Sur 842.0 km  área
8.80 km²  densidad 95.68 km/km² Z2 Nort
e          675.5 km  área 10.20 km²  dens
idad 66.23 km/km² Z4 Oriente 744.5 km
área 13.25 km²  densidad 56.19 km/km²

```

```

=== R3. Kilómetros por día y
meta de 500 km === Día 1:
540.0 km meta alcanza
da
Día 2: 430.5 km me
ta NO alcanzada Día 3:
515.5 km meta
alcanzada Día 4: 3
95.0 km meta NO alcan
zada Día 5: 560.0
km meta alcanzada
Día 6:500.0 km meta alcanzada

```

```

¿Meta alcanzada todos los días? No
¿Meta alcanzada al menos un día? Sí

```

=== R4. Liquidación semanal de repartidores ===

#	Cód.	Nombre	Km	Servicios	Bonific.	Alquiler	Neto
1	M01	Ana Torres	340.5	\$844.950	\$15.000	\$60.000	\$799.950
2	M03	Camila Restrepo	361.5	\$827.588	\$15.000	\$60.000	\$782.588
3	M09	Valeria Duque	352.0	\$842.513	\$0	\$60.000	\$782.513
4	M04	Juan Ríos	300.5	\$762.000	\$15.000	\$0	\$777.000
5	M07	Paula Cardona	344.0	\$813.063	\$0	\$60.000	\$753.063
6	M05	Laura Gómez	334.5	\$774.975	\$15.000	\$60.000	\$729.975
7	M02	David López	291.5	\$727.375	\$0	\$0	\$727.375
8	M06	Sergio Mejía	285.5	\$677.988	\$0	\$0	\$677.988
9	M08	Mateo Salazar	255.5	\$613.826	\$0	\$0	\$613.826
10	M10	Esteban Giraldo	75.5	\$203.850	\$0	\$0	\$203.850
11	M11	Daniela Ospina	0.0	\$0	\$0	\$0	\$0
12	M12	Tomás Arias	0.0	\$0	\$0	\$0	\$0

=== R5. Repartidor con más kilómetros cada día ===

Día 1: Laura Gómez (M05) con 90.0 km

Día 2: Camila Restrepo

(M03) con 80.0 km

Día 3: Camila Restrepo (M03) con

70.0 km

Día 4: Laura Gómez (M05) y Valeria

Duque (M09) con 71.0 km

Día 5: Paula Cardona (M07) con

79.5 km

Día 6: Juan Ríos (M04) con 80.5 km

Primer lugar en más días (2 días): Camila Restrepo (M03) y Laura Gómez (M05)

=== R6. Mejor puntualidad (retraso ponderado por km) ===

Mejor: David López (M02) con retraso ponderado de -2.16 min (11 servicios) Comparación de candidatos (ponderado vs promedio simple, en minutos):

David López (M02)	ponderado	-2.16	simple	-3.73	
Juan Ríos (M04)	ponderado	3.62	simple	3.40	
Ana Torres (M01)	ponderado	6.38	simple	5.60	
Paula Cardona (M07)	ponderado	11.57	simple	11.75	=
Sergio Mejía (M06)	ponderado	13.54	simple	12.30	
Mateo Salazar (M08)	ponderado	14.59	simple	13.80	=
Valeria Duque (M09)	ponderado	17.81	simple	26.08	
Camila Restrepo (M03)	ponderado	18.29	simple	19.64	=
Laura Gómez (M05)	ponderado	20.08	simple	20.27	

R7. Total pagado y costo por kilómetro == Total pagado en la semana: \$6.848.
128 Kilómetros totales:
2941.0 km
Costo promedio por km: \$2.329

== R8. Repartidores con servicios en todas las zonas == Ana Torres (M01)
David López (M02) Camila Restrepo (M03) Laura Gómez (M05) Sergio Mejía (M06) Paula Cardona (M07) Valeria Duque (M09)

Ingrese el código del repartidor para ver su comprobante:
== COMPROBANTE DE PAGO ==
Repartidor: Camila Restrepo Código: M03

Día	Km	Servicios	Bonificación
1	60.0	\$155.250	\$0
2	80.0	\$180.000	\$15.000
3	70.0	\$157.500	\$0
4	39.5	\$106.650	\$0
5	52.5	\$106.875	\$0
6	59.5	\$121.313	\$0

Suma valor de servicios: \$827.588
Suma de bonificaciones:

\$15.

000 Descuento por alquiler: \$60.

000 NETO A PAGAR: \$782.

588

=== Investigación C1: ranking/

2 con keyword lists === ranking

g(liq, campo: :neto, limite: 3)

1. Ana Torres: 799950

2. Camila Restrepo: 782588

3. Valeria Duque: 782513

ranking(liq, campo: :kilometros, orden: :asc, limite: 2)

1. Daniela Ospina: 0

2. Tomás Ari

as: 0 ranking(li

q, orden: :arriba)

-> :orden_i

nvalido ranking

g(liq, color: :ro

jo)

-> {:opciones_desconocidas, [:color]}

=== Investigación C2: Map.merge/3 ===

Propios (R3): % {1 => 540.0, 2 => 430.5, 3 => 515.5, 4 => 395.0, 5 => 560.0, 6 => 500.0}

Empresa aliada: % {1 => 580.5, 2 => 430, 3 => 510, 5 => 625, 7 => 180}

Map.merge/2: % {1 => 580.5, 2 => 430, 3 => 510, 4 => 395.0, 5 => 625, 6 => 500.0, 7 => 180}

Map.merge/3 (suma): % {1 => 1120.5, 2 => 860.5, 3 => 1025.5, 4 => 395.0, 5 => 1185.0, 6 => 500.0, 7 => 180}

=== Investigación C3: mediciones con :timer.tc/1 ===

1) Validar 116 servicios, una sola vez: 19 µs

- 2) Validar 1000 veces: 20236 μ s en total, 20.24 μ s por vez
- 3) Liquidar a todos 1000 veces: 34850 μ s en total, 34.85 μ s por vez
- 4) 100000 búsquedas de M12 en lista (Enum.any?): 23987 μ s
- 5) 100000 búsquedas de M12 en mapa (Map.has_key?): 8181 μ s

Pruebas de interacción con el usuario

Se probó el servicio adicional y el comprobante con entradas válidas e inválidas. En todos los casos el programa informa y continúa sin fallar.

Entrada	Respuesta del programa
(Enter)	No se ingresó ningún servicio adicional.
M03;Z2;4;22.5;-3	Servicio adicional agregado correctamente. (Se suma a todos los reportes: el día 4 pasa de 395,0 a 417,5 km.)
M03;Z2;4.0;22.5;-3	Rechazado por formato inválido (el día no es entero).
M03;Z2;4;abc;-3	Rechazado por formato inválido (kilómetros no numéricos).
M03;Z2;4;22.5	Rechazado por formato inválido (4 campos en vez de 5).
M03;Z2;4;22.5;-3;9	Rechazado por formato inválido (6 campos).
M99;Z2;4;22.5;-3	Rechazado por regla de validación: :repartidor_desconocido (aparece en R1).
M03;Z2;7;22.5;-3	Rechazado por regla de validación: :dia_invalido.
M03;Z2;4;50;-3	Rechazado por regla de validación: :kilometros_fuera_de_rango.
Código de comprobante: M03	Muestra el comprobante con los 6 días trabajados, sumas y neto.
Código de comprobante: M11	Muestra el comprobante sin días trabajados y todo en cero (no tiene servicios válidos).
Código de comprobante: ZZZ	No existe ningún repartidor con el código "ZZZ". El programa termina normalmente.

Parte D. Bitácora, caso de error y reflexión

Herramienta de IA utilizada: **Claude** (Anthropic), en la interfaz de chat. Fecha de las sesiones: 1 de octubre de 2026. Todas las respuestas fueron contrastadas ejecutando el programa y leyendo el código.

1. Bitácora de uso de IA

#	Qué se le pidió	Qué hizo / respondió la IA	Qué decidimos y por qué
1	Revisar el código terminado frente al enunciado del parcial.	Leyó los ocho archivos, ejecutó el programa y comparó la salida con las reglas. Concluyó que cumplía las reglas de validación, liquidación y los reportes R1–R8. Señaló tres defectos: los nombres de los integrantes sin completar, el costo por km de R7 con formato distinto al resto y finales de línea mezclados.	Aceptamos los tres hallazgos porque los verificamos en la salida y en los archivos; son correcciones de forma, no de lógica.
2	Contrastar el código con los ejercicios hechos en clase.	Indicó que el código de clase usa <code>destruct</code> (<code>Reserva.ex</code>) y recursividad (<code>Util2.ingresar_coleccion</code>), que el parcial prohíbe, y que nuestro código no los usa.	Mantuvimos la lectura de teclado propia (<code>Leer_linea</code>) y no usamos <code>Util2</code> en el parcial, para no heredar recursividad.
3	Revisar la coherencia de la entrada del usuario con el enunciado.	Notó que el servicio adicional pasaba el repartidor a mayúsculas, así que <code>m03</code> tecleado se aceptaba, mientras que <code>m01</code> en los datos se rechazaba por desconocido.	Quitamos la conversión a mayúsculas: la validación debe tratar igual un código venga de los datos o del teclado. Se probó de nuevo (<code>m03</code> ahora da <code>:repartidor_desconocido</code>).

4	Redactar este documento (Partes A, C, R6, D y salida).	Generó el borrador a partir del código real y de la salida real de la ejecución; calculó con nuestros datos el ejemplo de R6 (M09) y los resultados de las mediciones.	Revisamos que cada afirmación corresponda al código y que las cifras coincidan con la salida. Cada integrante debe poder explicarlo en la sustentación.
---	--	--	---

2. Caso de error

Error encontrado: formato inconsistente en R7. El programa corría sin fallas y los cálculos eran correctos, pero R7 mostraba el costo promedio por kilómetro como \$ 2 3 2 8 . 5 0 , mientras que en el resto de los reportes el dinero salía como \$ 6 . 8 4 8 . 1 2 8 (con punto de miles y sin decimales). La causa fue usar `Formato.decimal(costo, 2)` en lugar de `Formato.pesos/1`. Se corrigió con `Formato.pesos(round(costo))`, que ahora imprime \$ 2 . 3 2 9 . Lo aprendimos al leer la salida con calma: que un programa “corra” no significa que su salida sea correcta o consistente.

Riesgo con IA y código de clase: recursividad y structs. Los ejercicios del profesor incluyen `destruct` y una función recursiva. En esta revisión la IA no propuso ninguno de los dos, pero sabemos que las herramientas de IA suelen hacerlo aunque se les pida lo contrario. Por eso revisamos todo el código buscando `destruct`, `File`, `spawn`, `Task`, `Agent`, `GenServer`, `try y rescue`, y confirmamos que no aparecen. Los recorridos del programa son con `Enum` y `for`, incluida la repetición de las mediciones (`Enum.each(1..veces, ...)`).

3. Reflexión

La IA nos sirvió sobre todo para **verificar**: ejecutó el programa, probó entradas que nosotros no habíamos probado (por ejemplo, el día 4 . 0 o un código en minúsculas) y encontró incoherencias entre el enunciado y el código. También nos ahorró tiempo al armar este documento a partir de los datos reales. Pero no reemplaza entender el programa: no sabe por sí sola qué quiso decir el profesor, y su revisión solo es tan buena como las entradas que se le dan. Cada afirmación suya tuvimos que comprobarla nosotros.

En lo técnico, lo que más nos dejó el parcial fue ver por qué importa elegir bien la colección (mapa frente a lista, `Map` frente a `List` para R8, `keyword list` para opciones) y separar las funciones puras de las impuras: gracias a eso se pudo probar cada regla sin teclado. También entendimos que `with` y las tuplas `{:ok, _}` / `{:error, _}` reemplazan a `try/rescue` de forma más clara.

Nuestro compromiso para la sustentación es que cada integrante pueda explicar y modificar cualquier línea. Los puntos que más hay que dominar, por no haberse visto en clase, son: `with ... else`, `MapSet`, `Enum.frequencies` y `flat_map`, `Keyword.validate`, `Map.merge/3` y `timer.tc`.